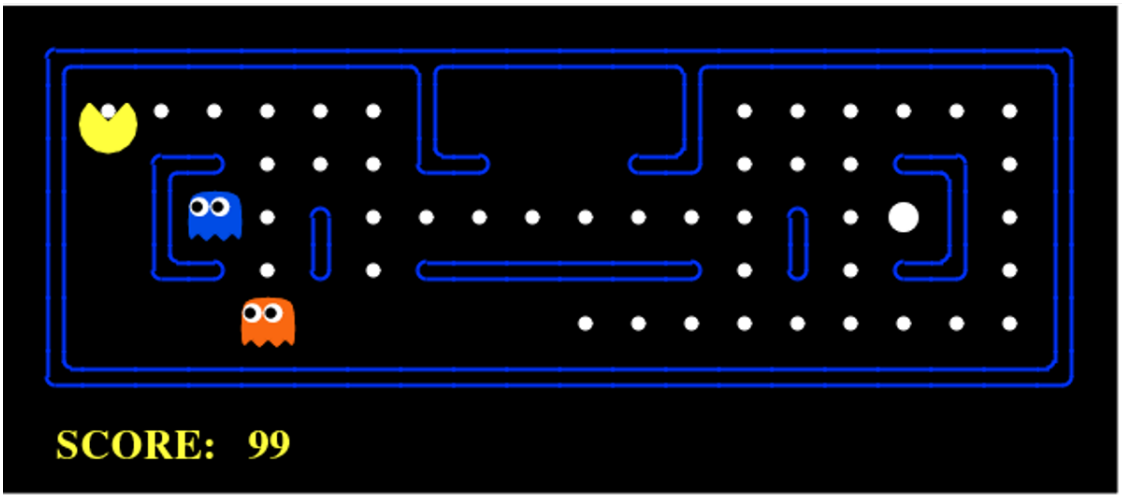
# FIT5047
# Assignment 1: Search




## Introduction

In the arcade game of *Pac-Man*, an eponymously-named agent accumulates points by collecting food dots while avoiding ghosts along the way. In the original version of the game, Pac-Man is controlled by a human player and ghosts are controlled by the environment. In this assignment, we will create a computer controller to play Pac-Man as efficiently as possible. We will consider different variations of the classic game to study different topics involving state-space search.

There are two parts to this assignment:

- **Part 1** is about single-agent search. You will tackle a number of different maze navigation challenges that ask you to collect dots as efficiently as possible.
- **Part 2** is about adversarial search. You need to program a computer controller that plays a real game of Pac-Man, ghosts and all!

This assignment is worth **25% of your marks** for this unit. Please check the marking rubric provided on Moodle for more details. You will be evaluated based on:

1. The performance of your controller on the various game challenges.
2. The quality of your report, which describes and justifies your implemented techniques.

The due date for this assignment is **August 28th, 2026 at 11:55pm** (Melbourne local time).

**SUBMISSION INSTRUCTIONS** for this assignment are described on the last page of this document. Please follow these instructions carefully to avoid disappointment.

# Pac-Man Overview

We will use a simulated version of the Pac-Man game, written entirely in Python and developed at the University of Berkeley. Instructions for how to download and install this software can be found in our **Installation Guide**, available on Moodle. Descriptions of the software and tips for how to tackle the assignment can be found in our **Getting Started Guide**, again available on Moodle.

In broad strokes:

1. The state of the game is tracked and advanced by a Pac-Man **game simulator**.
2. The simulator divides games into a sequence of discrete **iterations** or timesteps.
3. In each iteration, a **controller** object must specify valid actions for each agent in the game. The possible actions are: **Up, Down, Left, Right** and **Stop**.
4. Every action has a **cost of 1**.
5. Given a valid action, the game simulator **updates** the position of each agent and, if necessary, updates the score (invalid actions have no effect).
6. The game simulator continues iterating until one of the following **termination conditions** are reached:
    - The game board has been cleared of all food dots (i.e., the problem is solved).
    - Pac-Man is caught by a ghost (i.e., the game is over since Pac-Man only has a single "life").
    - Time runs out.

You will implement several Pac-Man controllers, each one solving a different kind of problem. Solution quality is measured in terms of **path length** (shorter is better). Each time you solve a problem, the corresponding **plan** (i.e., the sequence of actions computed by you) is printed to the terminal (stdout).

# Support

If you have questions about any part of this assignment specification, the starter code or its supporting documentation, please reach out to the teaching team. We suggest Ed as a first stop, followed by consultations and then email. We will do our best to respond to your questions as effectively as possible.

# Submission

**PAY CAREFUL ATTENTION TO THE INSTRUCTIONS IN THIS SPECIFICATION**

- Remember to submit all required files and documents!
- Do not modify any code that we do not ask you to!
- Each submission must be your own original work!

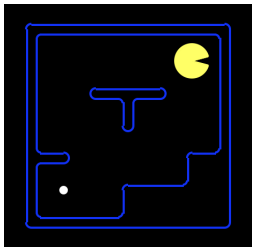
Failure to comply with these instructions may result in your attempt receiving an overall score of ZERO. Don't get caught out! Check and double check your attempt!

# Part 1: Single Agent Search

In this part of the assignment, we will develop an AI controller to play *Pac-Man: Collect the Dots*, a single-agent variant of Pac-Man. In this game, you must navigate a series of maze layouts, collecting as many dots as possible to maximise your score. There are no ghosts.

## Question 1(a)

In this question, **Pac-Man must find a path through the maze to collect a single dot**:



1. Define the problem (i.e., game state, etc.) in the `q1a_problem` class.
2. **Implement an A* algorithm** using the Manhattan distance heuristic in the `q1a_solver` class. Please only use the default Python modules, `numpy` and `scipy`. We will not be providing any others.

The command below shows how you can test your algorithm. Modify the `-l` parameter to try different maze layouts, which can be found at `layouts/q1a_*`.

```
> python pacman.py -l layouts/q1a_tinyMaze.lay -p SearchAgent -a
    fn=q1a_solver,prob=q1a_problem --timeout=1
```

### Marking

Your `q1a_solver` will be evaluated on 4 instances with a **timeout of 1 second**. **Timing out on any instance will result in 0 marks for that instance.**

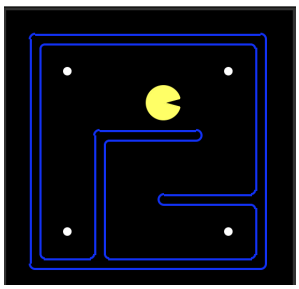
When your solver returns a plan for each instance, the game simulator will execute the actions one-by-one. The game finishes when there are no more actions to execute. If your Pac-Man agent fails to collect the dot, you will receive a score of 0 for this instance. If your Pac-Man successfully collects the dot, you will receive a mark of

$$\frac{C^*}{C}$$

where $C$ is the cost of your solution and $C^*$ is the minimum possible cost. We compute this number using a simple implementation of an A* algorithm. Full marks should be achievable for an adequate student. This mark is in the range $(0, 1]$ and each instance is worth a quarter of a mark, **allowing a maximum of 1 mark for your experimental results.**

## Question 1(b)

This question is similar to Q1(a) but there are now multiple dots in the maze. Your Pac-Man needs to collect one dot (you decide which) while minimising the total number of node expansions. Watch out for dots that are inaccessible!



1. Define the problem (i.e., game state, etc.) in the `q1b_problem` class.

2. **Modify your A* algorithm from Q1(a)** in the `q1b_solver` class. Define an efficient heuristic to compute a plan and guide the agent to the chosen dot.

You can run your code using the command below. Modify the `-l` parameter to try your solver on different maze layouts with the prefix `q1b_`.

```
> python pacman.py -l layouts/q1b_tinyCorners.lay -p SearchAgent -a
    fn=q1b_solver,prob=q1b_problem --timeout=5
```

### Marking

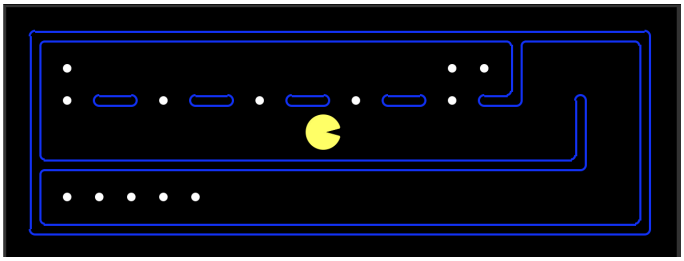
This task will be evaluated on 6 instances with a **time limit of 5 seconds**. **If Pac-Man fails to collect any dot, you will receive 0 marks for that instance.** Otherwise, you will be evaluated on the number of node expansions in your A* implementation. Your mark will be calculated as:

$$\min\left(\frac{\hat{E}}{E}, 1\right)$$

where $E$ is the number of node expansions in your algorithm and $\hat{E}$ represents the number of node expansions in a simple baseline that we implemented. Your mark for each instance ranges from 0 to 1 and each instance is worth half a mark, giving **a maximum of 3 marks for your experimental results.**

## Question 1(c)

This question is similar to Q1(b) but now you want to collect 'em all! Clear the board of every dot while keeping your path as short as you can. You will need to strategically balance the amount of search against the solution quality.



1. Define the problem (i.e., game state, etc.) in the `q1c_problem` class.

2. **Implement any algorithm** to solve this problem in the `q1c_solver` class.

The command below shows how you can test your implementation. Modify the `-l` parameter to try your solver on different maps with the prefix `q1c_`).

```
> python pacman.py -l layouts/q1c_tinySearch.lay -p SearchAgent -a
    fn=q1c_solver,prob=q1c_problem --timeout=10
```

### Marking

**There are 10 out of 100 marks for your computational results.** Your `q1c_solver` will be run on 12 evaluation instances. Each run has a **timeout of 10 seconds**. **Timing out on any of these instances will result in 0 marks for this instance.** Otherwise, we will calculate your mark based on the score shown in the game GUI, measured relative to the score achieved by a near-optimal tour of the map. The bonus for clearing the board counts as 150 points, i.e. we subtract 350 from the 500 points the simulator awards.

If you match or beat the benchmark score, then you will receive 1 (full) mark for that instance. Otherwise, you will receive a percentage based on the following formula:

$$\min\left(\max\left(\frac{S}{\hat{S}}, 0\right), 1\right)$$

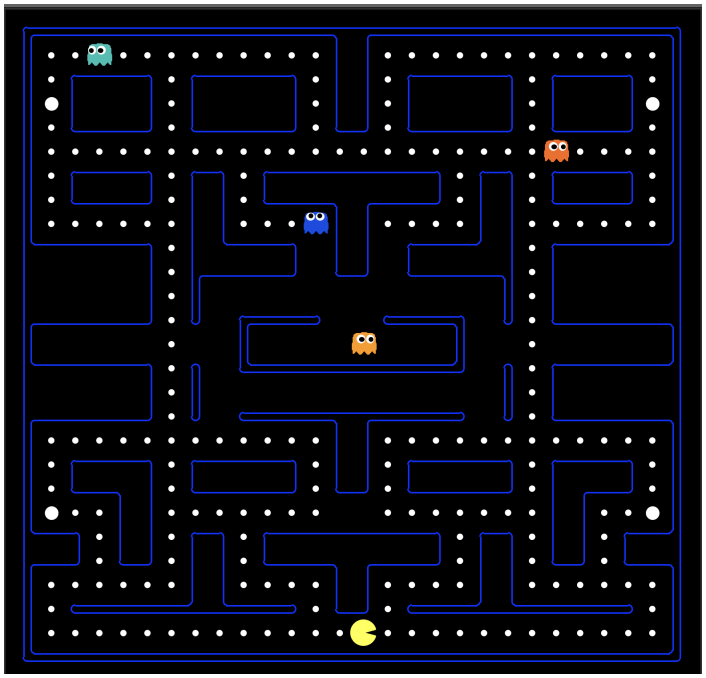
where $\hat{S}$ is the benchmark score and $S$ is your adjusted score. This formula restricts your grade to the range $[0, 1]$.

# Part 2: Adversarial Search

In this part of the assignment, you will develop a controller that plays an actual game of Pac-Man. You must collect dots for points but now there are ghosts to avoid.

## Question 2

There are multiple dots throughout the maze and 3-4 ghosts. The ghosts are controlled by the game environment.



- Implement the alpha-beta algorithm in the `getAction` function in the class `Q2_Agent`. During each iteration of the game, the `getAction` function will receive the current game state and perform an adversarial search. Pac-Man is the maximiser agent (i.e., alpha) and each ghost is a minimiser agent (i.e., beta). Once your search is finished, you should return the most promising action for Pac-Man to execute. You are free to customise your algorithm, provided that you base it on alpha-beta search.

You can use the command below to run your code. Modify the `-l` parameter to try different maze layouts located at `layouts/q2_*`.

```
> python pacman.py -l layouts/q2_testClassic.lay -p Q2_Agent --timeout=30
```

## Marking

**There are 41 out of 100 marks for your computational results.** Your `Q2_Agent` will be evaluated on 20 instances, each with a **time limit of 30 seconds**. This question is marked using the same criteria as question 1(c), except that we use the score in the GUI without any adjustment. Your mark is your score as a percentage of the score achieved by an existing algorithm but capped to the range $[0, 1]$. This algorithm has not been heavily optimised, therefore it should not be too difficult to achieve this.
**The top three agents that beat our algorithm in the full game will receive fame, glory and a certificate to document their superior skills at this game.**

## Report

You must write a report of **between 4-6 pages** that discusses what you learned, what you tried, what did work and what didn't work. We want to see that you have discovered and learnt something for yourself. If you tried different algorithms, this is the place to discuss them. You could also discuss and analyse the impact of the small scale, granular design choices in your implementation, such as data structures and tie breaking (if any). **The report is worth 45 out of 100 marks.**

## Hints and Tips

- Read the accompanying **Getting Started Guide**, a gentle introduction to the Pac-Man environment and tips for how to approach the assignment tasks.
- Lecture slides and tutorial exercises are good starting points for ideas. Check the recommended readings and optional resources for more ideas.
- Alternative text-books and related research papers (e.g., that you may find on Google Scholar) can provide you with further sources of inspiration.
- **Remember to justify your ideas in your report and to cite all your sources.**
- If you implement your stubs carefully, you may be able to re-use them to solve later search problems or to solve subproblems.

## Submission Instructions

1. Run the `evaluator.py` evaluation script to produce log files in the `logs` directory. Review these log files to confirm the correct execution of your program.
2. Your code is automatically graded online. Register at `http://fit5047.contest.pathfinding.ai/` using your Monash email address. You will then receive an email invitation to join a Git repository on GitHub.
3. **Edit the files described previously and push your code to GitHub. Make sure you do not use or edit any other file as we will not look at them.** If you're not sure how to **push** code to your git repository, please consult the resources linked in our Installation Guide.
4. In the website, click *Submit* and wait for the queue. We will clone your code from your Git repository and run the grading system. You can do this at any time before the deadline and there is no limit on how many times you can be graded. If there are errors, your will be notified. **We will retain the highest score for each of the four questions achieved across all submissions.**
5. **Upload your report on Moodle before the deadline.**
6. **Submissions after the deadline incur late penalties at a rate of 5% per day. After 7 days, your score is zero.**

## Plagiarism

You can be inspired by existing source code online but you need to acknowledge the source, discuss the approach in your report and then talk about your extensions/modifications.

**INTENTIONAL AND UNINTENTIONAL PLAGIARISM IS NOT ACCEPTABLE. YOU WILL FAIL THE ASSIGNMENT AND POSSIBLY THE ENTIRE UNIT.**

## Revision History

**1.3**

- Updated the due date.
- Reweighted the computational marks: 1 for Q1(a), 3 for Q1(b), 10 for Q1(c) and 41 for Q2.
- Q1(c) is graded on the length of your tour. The bonus for clearing the board counts as 150 rather than 500, and there are 12 evaluation instances instead of 10.
- Corrected the number of instances and ghosts in Q2.

**1.2**

- Updated for S1/2026
- Updated server URL (no vpn needed)

**1.1.1**

- Updated server URL

**1.1**

- Removed incorrect statement about saving log files as the default behaviour.

**1.0**

- First release of the assignment.